Trabajo Práctico 2 - Programación en C

Tecnología Digital II

Octubre 2026

1. Introducción

En la cátedra de TD2 hemos desarrollado un prototipo funcional del clásico juego de Tetris utilizando el lenguaje C y la librería SDL2 (ver `juego_base.c`). Si bien el código es jugable y cuenta con mecánicas, lógica de límites y renderizado, padece de serias limitaciones de diseño a nivel estructural.

El prototipo actual utiliza arreglos estáticos para representar los componentes principales del juego:

- La grilla se representa como `int grid[20][10]`. Al completarse una línea, se copian físicamente todas las celdas de cada línea superior una posición hacia abajo.
- La cola de próximas piezas (`QUEUE_SIZE`) y el sistema de partículas visuales (`MAX_PARTICLES`) están limitados por tamaños estáticos fijos. Con 15 partículas, por ejemplo, completar dos líneas a la vez ya descarta efectos.

El objetivo de este trabajo práctico es refactorizar estas estructuras utilizando el manejo manual de la memoria dinámica y estructuras de datos enlazadas. Se requiere que el tablero funcione como una *Lista Enlazada* de filas, de modo que eliminar una fila consista solamente en re-enlazar punteros, sin copiar celdas. Notar que, a cambio, acceder a la fila i pasa a requerir recorrer la lista (ver `getRowAt`). Tanto las piezas como las partículas deben responder a colas y listas cuyo tamaño no está fijado en tiempo de compilación.

Modalidad: El trabajo práctico debe realizarse en grupos de tres personas.

Fecha de Entrega: Martes 3 de noviembre hasta las 23:59hs.

2. Requisitos Técnicos

Para poder compilar y ejecutar el juego base y el esqueleto, es obligatorio instalar la librería SDL2 en el sistema operativo:

```
sudo apt-get install build-essential libsdl2-dev valgrind
```

Para compilar y ejecutar se utiliza el `Makefile` provisto, eligiendo en la variable `SRCS` el archivo a compilar:

```
make            # compila y genera el ejecutable ./tetris
make run        # compila y ejecuta el juego
make test       # ejecuta solo los tests (./tetris --test), sin abrir la ventana
make valgrind   # ejecuta los tests bajo valgrind
```

3. Enunciado

3.1. Ejercicio 1: Precalentamiento de Punteros sobre Strings

Previo a saltar a la lógica del juego, deberán implementar en su archivo las siguientes tres funciones clásicas operando sobre arrays de caracteres (strings). **No se permite el uso de la librería `string.h`**:

- `char* strDuplicate(char* src)`: Duplica un string en memoria. Debe contar la cantidad de caracteres de `src`, solicitar mediante `malloc` la memoria necesaria (incluyendo el terminador `'\0'`), y copiar todos los caracteres. Retorna el puntero al nuevo string.
- `int strCompare(char* s1, char* s2)`: Compara dos strings lexicográficamente. Devuelve 0 si son iguales, 1 si el primer string es menor al segundo, -1 en otro caso. **Atención:** es la convención inversa a la de `strcmp`. Por ejemplo, `strCompare("a", "b")` devuelve 1, `strCompare("b", "a")` devuelve -1, y `strCompare("casa", "casas")` devuelve 1 (un prefijo es menor).
- `char* strConcatenate(char* src1, char* src2)`: Retorna un nuevo string dinámico que contiene una copia de todos los caracteres de `src1` seguidos de `src2`. Además, la memoria original apuntada por `src1` y `src2` debe ser liberada internamente utilizando `free`. Por lo tanto, ambos argumentos deben haber sido obtenidos con `malloc` (por ejemplo, mediante `strDuplicate`) y ser punteros distintos: pasar un literal como `"abc"`, o el mismo puntero dos veces, produce un `free` inválido.

3.2. Ejercicio 2: Refactorización del Tetris

Partiendo del archivo `solucion_esqueleto.c`, el cual ya cuenta con las definiciones `struct` de `RowNode`, `PieceQueueNode` y `ParticleNode`, deberán completar el interior de las siguientes funciones para gestionar la memoria de los nodos:

- `GameBoard* gameBoardNew()`: Reserva la estructura principal e inicializa todos sus campos (punteros en `NULL`, `score` y `tick_timer` en 0, sin pieza activa). Debe además alojar dinámicamente (`malloc`) los 20 nodos iniciales de las filas (`RowNode`), con todas sus celdas en 0, y conectarlos entre sí estableciendo el puntero `first_row` a la cima. Si algún `malloc` falla, debe liberar lo ya reservado y devolver `NULL`.
- `void gameBoardDelete(GameBoard* board)`: Libera **toda** la memoria dinámica (filas, piezas de la cola, partículas y el tablero en sí).
- `void gameBoardEnqueuePiece(GameBoard* board, int type)`: Lógica FIFO de inserción al final de la lista (`queue_tail`).
- `int gameBoardDequeuePiece(GameBoard* board, int* out_type)`: Extracción de la cabeza de la lista FIFO, asignando el valor al puntero saliente. Devuelve 1 si extrajo una pieza y 0 si la cola estaba vacía (en ese caso no modifica `*out_type`). Si la cola queda vacía, tanto `queue_head` como `queue_tail` deben quedar en `NULL`.
- `void gameBoardAddParticle(GameBoard* board, float x, float y, int color)`: Crea un nodo `ParticleNode` y lo añade como cabeza de la lista simple de partículas.
- `int gameBoardCheckAndClearLines(GameBoard* board)`: Recorre la lista enlazada de filas. Cuando encuentra un nodo donde ninguna celda es 0, genera una partícula por celda (`gameBoardAddParticle`), desenlaza dicho nodo modificando los punteros adyacentes, hace `free()`, y regenera una fila en blanco que se inserta arriba del todo en `board->first_row`. Devuelve la cantidad de líneas eliminadas. Al terminar, el tablero debe seguir teniendo exactamente 20 filas, incluso si se completan varias líneas a la vez o si la línea completa es la primera de la lista.

4. Casos de Test Mínimos

Se deben incluir los siguientes casos de test, implementados en la función `void correrTests()` del esqueleto, que se ejecuta antes de inicializar SDL. Ejecutando `./tetris --test` (o `make test`) se corren solo los tests, sin abrir la ventana. Así `valgrind` reporta únicamente la memoria del TP, ya que SDL mantiene bloques propios que no libera.

strDuplicate: 1. String vacío. 2. String de un carácter. 3. String con distintos caracteres.

strCompare: 1. Dos strings vacíos. 2. Dos strings de un carácter. 3. Dos strings distintos, comparados en ambos órdenes (s1 vs s2 y luego s2 vs s1).

strConcatenate: 1. String vacío y string de 3 caracteres. 2. Dos strings de 5 caracteres. Los argumentos deben crearse con `strDuplicate`. (Asegurar no tener Memory Leaks verificando con *valgrind*).

Lógica de Tablero:

1. Hacer Push a la Queue 5 veces y Pop 6 veces, validando el orden FIFO y que el sexto Pop devuelva 0 dejando `queue_head` y `queue_tail` en `NULL`. Luego hacer un Push más y verificar que se puede volver a extraer.

2. Llenar artificialmente la última fila (índice 19, contando desde 0). Luego, invocar `gameBoardCheckAndClearLines` y comprobar que devuelve 1, que la fila recién creada vacía es ahora `first_row` y que la lista sigue teniendo 20 filas.

3. Llenar las filas 0 y 1 y llamar a `gameBoardCheckAndClearLines`, comprobando que devuelve 2 y que la lista sigue teniendo 20 filas.

5. Entregable

La entrega consiste en un link a un repositorio de Github donde se encuentre el archivo fuente y el makefile correspondiente para poder ejecutar el trabajo. No se debe entregar un informe en PDF, pero sí se exige colocar comentarios técnicos concisos sobre las decisiones de uso de `malloc` y lógica de punteros aplicadas a las listas.

El código debe compilar sin warnings con `-Wall`, no debe usar `string.h`, y `make valgrind` no debe reportar memoria perdida ni accesos inválidos.